AI-DRIVEN SOFTWARE ENGINEERING & DEVOPS

IV B.Tech I Semester — Professional Elective IV

UNITS III, IV AND V

Exam Preparation Guide

For each unit: 5 detailed 10-mark answers and 10 short 2-mark answers.

Prepared from the syllabus topics shown in the provided course outline.

UNIT III: Intelligent Testing, QA, and Debugging

PART A — 10-MARK QUESTIONS WITH DETAILED ANSWERS

Q1. Explain test-case generation using AI and its advantages in software testing.

Introduction: AI-based test-case generation uses Artificial Intelligence and Machine Learning to create test cases from requirements, source code, application behavior, and historical test data.

1. Working process
1. Input collection: AI analyzes requirements, source code, APIs, and existing tests.
2. Program analysis: It identifies functions, conditions, execution paths, and possible error scenarios.
3. Test generation: It creates test inputs and expected outcomes where these can be determined.
4. Test execution: Generated tests are run against the software.
5. Result analysis: Actual results are compared with expected behavior.
6. Test improvement: Results and uncovered code paths help refine subsequent tests.

2. Techniques used
- Machine Learning learns patterns from historical defects and test results.
- Natural Language Processing (NLP) converts written requirements into candidate scenarios.
- Search-based testing searches for inputs that satisfy testing goals.
- Symbolic execution analyzes program conditions to derive inputs for particular paths.
- Generative AI suggests test code, test data, and edge cases from code or specifications.

3. Types of test cases
- Normal cases for valid inputs.
- Boundary-value cases.
- Invalid-input cases.
- Exception-handling cases.
- Integration and API cases.
- Regression test cases.

Example: For a function accepting marks from 0 to 100, AI-assisted testing may suggest 0, 1, 50, 99, 100, -1, and 101 to test normal, boundary, and invalid conditions.

4. Advantages
- Reduces manual test-writing effort.
- Increases test coverage.
- Helps identify unusual input combinations.
- Speeds up testing cycles.
- Supports continuous integration.
- Helps discover defects earlier.

5. Challenges
Generated tests may contain incorrect assertions; AI may misunderstand requirements; some expected outputs are difficult to determine automatically; and generated tests require review and maintenance.

Conclusion: AI-based test-case generation improves testing speed and coverage. The best results come from combining automated generation with human review and reliable expected results.

Q2. Explain automated unit testing and regression testing using Machine Learning.

Introduction: Unit testing verifies individual software components, while regression testing ensures existing functionality continues to work after code changes. AI and ML can improve both by generating tests and prioritizing important test cases.

1. Automated unit testing
Unit testing checks individual functions, methods, or modules independently. AI can assist with generating test code, identifying input conditions, suggesting assertions, detecting boundary cases, finding missing coverage, and updating tests when code changes.

Example: For a function calculating the area of a rectangle, tests should verify positive dimensions, zero dimensions, and invalid negative dimensions according to the requirements.

2. Regression testing
Regression testing is performed after modifications to determine whether existing features have been affected. For example, modifying a payment module might accidentally affect order totals or invoice generation.

3. Role of Machine Learning
ML models can analyze previous test failures, changed source files, dependencies between modules, defect history, test execution times, and code complexity. They can rank test cases by their likelihood of detecting defects.

4. Regression-test prioritization
1. Identify modified files.
2. Determine dependent modules.
3. Analyze historical failures.
4. Rank relevant test cases.
5. Execute high-priority tests first.
6. Run the remaining required tests.
7. Review results and update test history.

5. Benefits
Faster feedback, early detection of regressions, reduced unnecessary test execution, better use of computing resources, and improved reliability of frequent releases.

6. Limitations
Historical data may not represent new defects; incorrect predictions can rank important tests too low; test suites still need adequate coverage; and generated tests may fail to check the correct behavior.

Conclusion: AI-enhanced unit and regression testing can make testing more efficient. Prioritization must not replace essential testing, especially for safety-critical or security-sensitive software.

Q3. Explain bug-pattern identification, AI-based static and dynamic code analysis, and fault localization.

Introduction: Software defects can occur because of incorrect logic, invalid inputs, resource-management problems, or security weaknesses. AI helps developers learn defect patterns, examine program behavior, and locate suspicious code.

1. Bug-pattern identification
It detects recurring coding mistakes by analyzing source code, defect reports, and repository history. Common examples include null-pointer errors, incorrect conditional statements, memory leaks, resource-handling errors, race conditions, and security vulnerabilities.

2. Static code analysis
Static analysis examines source code without executing it. AI-enhanced static analysis may identify suspicious code structures, unsafe operations, duplicated code, complex functions, security weaknesses, and coding-standard violations.

3. Dynamic code analysis
Dynamic analysis examines a program while it executes. It can detect runtime exceptions, memory problems, unexpected output, performance bottlenecks, and incorrect component interactions. AI can correlate execution traces and runtime logs.

4. Fault localization
Fault localization attempts to identify the statement, function, or module responsible for a test failure.
1. Execute the test suite.
2. Collect passing and failing results.
3. Analyze execution traces and logs.
4. Compare code exercised by passing and failing tests.
5. Rank suspicious statements or functions.
6. Verify the suspected defect and implement a fix.

Example: If several tests fail after a change to a shared validation function, an AI-assisted tool may rank that function highly because it is associated with many failing paths.

5. Benefits
Earlier defect detection, faster debugging, better code quality, less time searching large codebases, and improved security review.

6. Limitations
Static analysis may produce false positives; dynamic analysis may miss untested paths; and fault-localization results identify suspicious code rather than guaranteed causes.

Conclusion: AI-powered bug identification, code analysis, and fault localization reduce debugging effort and help developers build more reliable software.

Q4. Explain automated debugging and AI-based quality assurance metrics.

Introduction: Automated debugging uses software tools to identify, diagnose, and sometimes correct defects. AI-based quality assurance (QA) enhances this process by analyzing code, test results, and defect trends.

1. Automated debugging
Activities include detecting errors from logs and test failures, identifying suspicious functions, explaining error messages, suggesting corrections, generating candidate patches, and running tests to verify fixes.

2. AI-assisted debugging process
1. A defect is detected during testing or monitoring.
2. Logs, stack traces, and relevant source code are collected.
3. AI analyzes the evidence and identifies possible causes.
4. A developer examines the suggested diagnosis.
5. A correction is implemented.
6. Automated tests verify the correction.
7. The change is reviewed before release.

Example: If a program crashes because it accesses a missing value, an AI tool may identify the relevant expression and suggest input validation or appropriate null handling.

3. QA metrics
- **Code coverage:** Percentage of code exercised by tests.
- **Defect density:** Number of defects relative to software size.
- **Test pass rate:** Percentage of executed tests that pass.
- **Defect detection rate:** Rate at which defects are discovered.
- **Mean time to repair (MTTR):** Average time to repair or restore a failed system.
- **Escaped defects:** Defects discovered after release.

4. Role of AI
AI analyzes historical metrics to identify trends, predict high-risk areas, detect unusual changes, and prioritize quality improvements.

5. Advantages
Faster defect diagnosis, less repetitive debugging, better visibility into quality trends, earlier identification of risky modules, and improved feedback.

6. Challenges
AI fixes may introduce new defects; metrics may be misleading without context; high coverage does not guarantee correct software; and log interpretation can be wrong.

Conclusion: AI-based debugging and QA metrics improve diagnosis and monitoring. Fixes should be verified using tests, code review, and security checks.

Q5. Discuss reinforcement learning for test prioritization, and explain ethics and bias in AI-driven testing.

Introduction: Reinforcement Learning (RL) is a Machine Learning approach in which an agent learns to choose actions based on rewards or penalties. In software testing, RL can help optimize test execution order and resource allocation.

1. Reinforcement learning components
- **Agent:** System choosing actions.
- **Environment:** Testing system.
- **State:** Current information, such as recent failures and coverage.
- **Action:** Selecting the next test or testing strategy.
- **Reward:** Feedback based on the chosen action.

2. RL for test prioritization
1. Observe code changes and previous test results.
2. Select a test to execute.
3. Receive the test result.
4. Obtain a reward based on the defined objective.
5. Update the policy to improve future test selection.

A reward may favor early detection of important defects while accounting for execution time.

3. Benefits
Adapts to changing software behavior, helps prioritize tests under time constraints, improves repeated test cycles, and uses feedback to refine decisions.

4. Challenges
Reward design is difficult; historical results may be incomplete; the model may over-prioritize familiar defects; and required tests must not be skipped merely to optimize speed.

5. Ethics in AI-driven testing
- **Bias:** Training data may underrepresent particular inputs or user groups.
- **Privacy:** Test data may contain personal or confidential information.
- **Transparency:** Developers should understand important recommendations.
- **Accountability:** Teams must define who reviews and approves AI-generated changes.
- **Reliability:** AI can generate incorrect tests or miss defects.

6. Risk reduction
Use representative test data, remove unnecessary personal information, evaluate models on unseen examples, maintain human oversight, record decisions, and verify generated tests and fixes independently.

Conclusion: RL can improve test prioritization, but it must be combined with reliable testing requirements, transparent evaluation, and ethical safeguards.

PART B — 10 TWO-MARK QUESTIONS WITH ANSWERS

1. What is AI-based software testing?

AI-based software testing uses AI and ML to generate test cases, analyze results, detect defects, and improve software quality.

2. What is test-case generation?

It is the process of creating inputs, execution conditions, and expected results to verify software behavior.

3. Define unit testing.

Unit testing verifies individual functions, methods, or modules independently to identify defects early.

4. What is regression testing?

Regression testing checks whether software changes have affected previously working functionality.

5. What is static code analysis?

Static code analysis examines source code without executing it to identify possible defects and security weaknesses.

6. What is dynamic code analysis?

Dynamic code analysis examines program behavior during execution to identify runtime errors and performance issues.

7. Define fault localization.

Fault localization identifies or ranks parts of a program that are likely to cause a failure.

8. What is automated debugging?

Automated debugging uses tools to analyze failures, identify possible causes, and suggest or verify fixes.

9. What is code coverage?

Code coverage measures how much of a program's code is exercised by a test suite.

10. What is reinforcement learning in software testing?

It is an ML approach in which a system learns testing strategies through actions and feedback, such as rewards for finding defects efficiently.

UNIT IV: AI in DevOps Automation and CI/CD

PART A — 10-MARK QUESTIONS WITH DETAILED ANSWERS

Q1. Explain DevOps fundamentals and the integration of AI into the DevOps lifecycle.

Introduction: DevOps is a software development and operations approach emphasizing collaboration, automation, continuous feedback, and reliable software delivery. AI enhances DevOps by predicting problems, analyzing operational data, and supporting automated decisions.

1. DevOps fundamentals
- **Collaboration:** Development and operations teams share responsibility.
- **Automation:** Repetitive building, testing, and deployment tasks are automated.
- **Continuous Integration:** Code changes are integrated and validated frequently.
- **Continuous Delivery:** Software is kept ready for reliable release.
- **Infrastructure as Code:** Infrastructure is managed using configuration files.
- **Continuous Monitoring:** Application and infrastructure behavior is monitored.
- **Continuous Feedback:** Operational information improves future releases.

2. AI integration in DevOps
1. Planning: Analyze historical work and identify delivery risks.
2. Coding: Assist with code completion, explanation, and review.
3. Building: Predict build failures from previous results.
4. Testing: Generate or prioritize test cases.
5. Releasing: Estimate release risks and analyze deployment history.
6. Deploying: Monitor rollout health and support rollback decisions.
7. Operating: Detect anomalies in logs, metrics, and traces.
8. Monitoring: Predict incidents and identify unusual resource consumption.

3. Example
A team releasing a web application daily can use AI to analyze previous build failures, identify risky changes, prioritize related tests, and monitor response times after deployment. Serious errors can trigger a rollback under predefined rules.

4. Advantages
Faster release cycles, reduced repetitive effort, earlier failure detection, improved reliability, better resource use, and improved feedback.

5. Challenges
Integration with existing tools, poor-quality monitoring data, false alarms, inaccurate predictions, security risks, and excessive trust in automated recommendations.

Conclusion: AI makes the DevOps lifecycle more data-driven and proactive while retaining human oversight for important decisions.

Q2. Explain intelligent CI/CD pipeline design and predictive build-failure analysis.

Introduction: A CI/CD pipeline automates the steps required to build, test, and release software. AI improves pipeline efficiency by analyzing historical data, predicting failures, prioritizing tests, and monitoring deployments.

1. CI/CD pipeline stages
1. Source: Developers commit code to a repository.
2. Build: The system compiles and packages the application.
3. Test: Automated tests check functionality and quality.
4. Analyze: Code-quality and security tools examine changes.
5. Release: Software is prepared for deployment.
6. Deploy: The application is released to a target environment.
7. Monitor: Logs, metrics, and traces are analyzed.
8. Feedback: Results improve future pipelines.

2. Intelligent pipeline design
AI examines code changes and recommends tests to run first. It can identify risky dependencies and detect unusual build durations.

3. Predictive build-failure analysis
It uses historical build records and ML models to estimate whether a new build is likely to fail. Input features may include changed files, size of the code change, recent build failures, test failure history, dependency modifications, code complexity, and defects in affected modules.

4. Working process
1. Collect historical build and test data.
2. Clean and prepare the dataset.
3. Select useful features.
4. Train and evaluate an ML model.
5. Analyze a new code change.
6. Estimate failure risk.
7. Alert developers or prioritize additional checks.
8. Compare predictions with outcomes and update the model.

5. Example
If changes to a shared authentication module frequently caused failed builds, a model may flag a new change to that module as high risk. The pipeline can run additional security and integration tests.

6. Advantages
Earlier identification of risky changes, reduced wasted build time, faster feedback, improved release planning, and better allocation of testing resources.

7. Limitations
Predictions are estimates, not guarantees. New defects may not resemble historical failures. Low predicted risk should not bypass essential tests.

Conclusion: Intelligent CI/CD combines automated delivery with predictive analytics to detect risks early and improve release reliability.

Q3. Explain AI in Infrastructure as Code, deployment orchestration, and self-healing systems.

Introduction: Infrastructure as Code (IaC) manages computing resources using machine-readable configuration files. AI can help analyze infrastructure configurations, coordinate deployments, detect failures, and recommend recovery actions.

1. Infrastructure as Code
IaC describes infrastructure such as servers, networks, storage, and cloud services in configuration files. Examples of IaC tools include Terraform and Ansible. AI can detect configuration errors, identify insecure permissions, find unnecessary resources, suggest improvements, and review changes.

2. Deployment orchestration
Deployment orchestration coordinates application releases. AI can help select deployment windows, identify risky releases, analyze health during rollout, recommend canary or staged deployment, and support rollback decisions.

3. Common deployment strategies
- **Rolling deployment:** Replaces application instances gradually.
- **Blue-green deployment:** Maintains two environments and switches traffic when the new version is ready.
- **Canary deployment:** Releases a new version to a small group before expanding rollout.
AI can analyze error rates and response times to evaluate deployment health.

4. Self-healing systems
A self-healing system detects certain failures and performs recovery actions automatically. Examples include restarting an unhealthy service, replacing a failed instance, scaling resources, reverting a release after failed health checks, or redirecting traffic away from an unhealthy component.

5. Working process
1. Monitor system health.
2. Detect an abnormal condition.
3. Diagnose the likely problem.
4. Select an approved recovery action.
5. Execute the action.
6. Verify service recovery.
7. Escalate to a human if recovery fails.

6. Advantages
Reduced downtime, faster incident recovery, consistent configuration, better resource use, and less manual intervention.

7. Challenges
Incorrect automated actions can worsen incidents. Recovery systems need access controls, action limits, audit logs, health verification, and rollback mechanisms.

Conclusion: AI-assisted IaC, orchestration, and self-healing improve reliability when recovery actions are carefully controlled.

Q4. Explain log analytics, anomaly detection, predictive monitoring, and feedback loops in DevOps.

Introduction: Modern software systems produce large amounts of logs, metrics, and distributed traces. AI analyzes these signals to detect abnormal behavior, predict incidents, and improve operational decisions.

1. Log analytics
Log analytics examines records from applications, servers, databases, and networks. AI can group similar messages, identify recurring errors, detect unusual event sequences, correlate errors across services, and help identify probable root causes.

2. Anomaly detection
Anomaly detection identifies behavior that differs from expected patterns. Examples include sudden increases in CPU usage, unexpected memory consumption, spikes in failed login attempts, slow API responses, and increased database errors. Methods include statistical thresholds, clustering, and supervised or unsupervised ML.

3. Predictive monitoring
Predictive monitoring uses historical and current observations to estimate future problems. For example, if disk usage rises steadily, a model may estimate when storage could become insufficient.

4. Distributed tracing
Distributed tracing records a request as it travels across services. AI can help identify which service or dependency contributes most to latency or failure.

5. Feedback loops
1. Deploy software.
2. Collect logs, metrics, traces, and user feedback.
3. Analyze performance.
4. Identify defects or bottlenecks.
5. Update tests, configurations, or code.
6. Deploy the improvement.
7. Monitor results.

6. Benefits
Faster incident detection, better root-cause investigation, reduced downtime, informed capacity planning, and continuous improvement.

7. Challenges
Noisy logs, changing traffic patterns, incomplete data, and false alarms can reduce model accuracy. Monitoring data must also be protected from unauthorized access.

Conclusion: AI-powered monitoring and feedback loops make DevOps proactive by converting operational data into predictions and improvements.

Q5. Explain AI for DevSecOps and security automation in the software delivery pipeline.

Introduction: DevSecOps integrates security into development, testing, deployment, and operations. AI supports it by identifying suspicious code, prioritizing vulnerabilities, analyzing logs, and automating selected security tasks.

1. Security throughout the lifecycle
Security should be checked throughout the pipeline rather than only after deployment.

2. AI-assisted source-code analysis
AI tools can identify suspicious code patterns and suggest improvements. Static Application Security Testing (SAST) examines source code for potential vulnerabilities.

3. Dependency analysis
Software uses external libraries. AI-assisted tools can help identify vulnerable dependencies and prioritize updates using vulnerability severity and application context.

4. Dynamic security testing
Dynamic Application Security Testing (DAST) examines a running application for security weaknesses. AI may assist with test selection and result analysis.

5. Anomaly detection
AI can analyze authentication events, network traffic, access logs, and application behavior to identify suspicious activity.

6. Security automation in CI/CD
1. Code commit and repository checks.
2. Secret scanning.
3. Static code analysis.
4. Dependency vulnerability scanning.
5. Automated functional and security testing.
6. Deployment approval based on risk.
7. Runtime monitoring.
8. Incident response and feedback.

7. Benefits
Earlier discovery of vulnerabilities, faster security feedback, better risk prioritization, less repetitive work, and improved visibility into software supply-chain risks.

8. Challenges and safeguards
AI may miss vulnerabilities or generate false alarms. Security controls should use trusted scanners, current vulnerability information, human review for high-impact findings, least-privilege access, and secure handling of code and logs.

Conclusion: AI-enhanced DevSecOps makes security continuous. AI assists but does not replace security engineering, rigorous testing, or accountability.

PART B — 10 TWO-MARK QUESTIONS WITH ANSWERS

1. What is DevOps?

DevOps combines development and IT operations through collaboration, automation, continuous integration, delivery, and monitoring.

2. Define Continuous Integration (CI).

CI is the practice of frequently merging code changes into a shared repository and automatically building and testing them.

3. What is Continuous Delivery?

Continuous Delivery keeps software in a deployable state through automated building, testing, and release preparation.

4. What is predictive build-failure analysis?

It uses historical build data and ML models to estimate whether a new build is likely to fail.

5. Define Infrastructure as Code (IaC).

IaC manages infrastructure using machine-readable configuration files rather than relying entirely on manual configuration.

6. What is AIOps?

AIOps applies AI and analytics to IT operations to detect anomalies, predict failures, and support automated incident management.

7. What is log analytics?

Log analytics examines application and infrastructure logs to identify errors, patterns, unusual events, and possible root causes.

8. Define anomaly detection.

Anomaly detection identifies observations that differ significantly from expected system behavior.

9. What is a self-healing system?

A self-healing system detects certain failures and automatically performs recovery actions, such as restarting an unhealthy service.

10. What is DevSecOps?

DevSecOps integrates security practices into the development and operations lifecycle, including automated security checks in CI/CD pipelines.

UNIT V: Advanced Topics and Ethical Considerations

PART A — 10-MARK QUESTIONS WITH DETAILED ANSWERS

Q1. Explain explainability and transparency in AI-driven software engineering.

Introduction: Explainability and transparency are important characteristics of trustworthy AI systems. They help developers understand AI recommendations, evaluate reliability, and determine whether output should be accepted.

1. Explainability
Explainability is the ability to understand why an AI system produced a particular prediction, classification, or recommendation. For example, an AI code-review tool might flag a function because of high complexity, missing input validation, and similarity to previously defective code.

2. Transparency
Transparency means providing appropriate information about an AI system's operation, limitations, data, and use. It may include model purpose, data sources, evaluation results, model version, known failure cases, and human review procedures.

3. Difference
- **Explainability:** Focuses on why a particular output was produced.
- **Transparency:** Focuses on making the system's design, operation, limitations, and use understandable.

4. Importance
These properties help with debugging incorrect AI-generated code, reviewing test recommendations, investigating security alerts, understanding defect-risk predictions, auditing decisions, and building trust.

5. Techniques
- Feature importance identifies influential input features.
- Example-based explanations show similar cases.
- Rule-based explanations describe decisions using understandable rules.
- Visualizations show model behavior or influential variables.
- Traceability links outputs to inputs, code, and tool versions.

Not every model can provide a complete or perfectly faithful explanation, so explanations must be evaluated.

6. Example
An AI tool predicts that a module has high defect risk. It may identify high complexity, frequent recent changes, and a history of failed tests as contributing factors, allowing developers to investigate them.

7. Challenges
Complex models can be difficult to explain; explanations may be incomplete or misleading; sensitive code may be exposed; and different stakeholders need different levels of detail.

Conclusion: Explainability helps people understand individual outputs, while transparency helps them understand the broader system. Both support responsible AI engineering.

Q2. Discuss the ethical and legal aspects of AI in automated software engineering.

Introduction: AI tools can generate code, review software, create tests, and automate engineering workflows. Their use raises ethical and legal questions involving fairness, privacy, accountability, intellectual property, and security.

1. Ethical aspects
- **Bias and fairness:** Unrepresentative data may lead to biased outputs; testing should include relevant inputs and user groups.
- **Privacy:** Source code, logs, and test data may contain confidential or personal information; minimize data and apply access controls.
- **Accountability:** Teams must define responsibility for reviewing, approving, and maintaining AI-generated code.
- **Reliability:** AI may produce incorrect code or incomplete security checks; outputs must be tested.
- **Human oversight:** High-impact decisions need appropriate supervision and escalation procedures.

2. Legal aspects
- Data protection and privacy requirements.
- Copyright and intellectual-property risks.
- Software licensing obligations.
- Cybersecurity requirements.
- Contracts and restrictions for external AI services.
- Industry- and jurisdiction-specific regulation.

3. Risk-reduction practices
1. Establish an AI usage policy.
2. Use approved tools for confidential code.
3. Review licenses and third-party dependencies.
4. Perform security testing on generated code.
5. Document significant AI-assisted decisions.
6. Define responsibility for approvals and incidents.
7. Monitor compliance as tools and requirements change.

4. Example
If a developer submits AI-generated code with an insecure authentication mechanism, the organization may face security and contractual risks. Code review, security scanning, testing, and documented approval help reduce those risks.

5. Benefits of responsible AI use
Better protection of sensitive information, improved fairness and reliability, clearer accountability, reduced licensing and compliance risks, and greater confidence in AI-assisted development.

Conclusion: Ethical and legal considerations should be integrated into the AI lifecycle. Compliance requires context-specific review, not merely a claim that a tool is safe.

Q3. Explain risk management in autonomous code deployment and AI-based decision-making.

Introduction: Autonomous code deployment releases software with limited manual intervention. AI may estimate deployment risks and recommend actions, but incorrect decisions can cause outages, data loss, or security incidents. Risk management reduces these possibilities.

1. Meaning
Risk management is the systematic process of identifying, evaluating, prioritizing, treating, and monitoring risks.

2. Risk identification
Potential risks include incorrect AI-generated code, failed builds or tests, security vulnerabilities, incompatible dependencies, unexpected performance changes, data corruption, and deployment failures.

3. Risk assessment
Each risk can be assessed based on likelihood and potential impact. A qualitative matrix classifies risks as low, medium, or high. More formal methods may use numerical estimates when reliable data is available.

4. Risk mitigation
Use automated unit and integration tests, security and dependency scans, approval for high-risk changes, staged or canary deployments, health checks, rollback mechanisms, access control, and audit logs.

5. Controlled autonomous deployment
1. Generate or modify code.
2. Build and test the application.
3. Evaluate security and quality checks.
4. Estimate release risk.
5. Require approval where policy demands it.
6. Deploy to a limited environment.
7. Monitor errors and performance.
8. Expand the release if health checks pass.

9. Stop or roll back if serious problems occur.

6. AI for technical risk prediction
AI may analyze code complexity, historical incidents, dependency changes, and previous release failures to flag risky changes. Predictions should inform decisions rather than replace essential quality gates.

7. Monitoring and incident response
Risk management continues after deployment. Teams must monitor system health, investigate incidents, document failures, and improve controls.

8. Advantages
Reduced likelihood of major failures, faster detection and recovery, better release decisions, consistent deployment procedures, and improved accountability.

Conclusion: Risk management makes AI-assisted deployment safer through testing, controlled releases, monitoring, human oversight, and reliable recovery mechanisms.

Q4. Explain human–AI collaboration in software teams and AI for technical debt prediction.

Introduction: Human–AI collaboration combines human judgment and domain expertise with AI's ability to analyze data and generate suggestions. AI can also help identify technical debt so teams can plan maintenance more effectively.

1. Human–AI collaboration
AI tools can assist with code generation and completion, documentation, test-case creation, code review, debugging, refactoring suggestions, and searching technical documentation.

2. Collaboration workflow
1. A developer defines the problem.
2. AI proposes a solution.
3. The developer evaluates the proposal.
4. Tests and security checks are performed.
5. The developer modifies or approves the result.
6. Code is integrated and monitored.

Example: AI generates a function for validating user input. The developer reviews conditions, checks edge cases, tests invalid input, and confirms the function meets requirements.

3. Benefits
Faster routine development, reduced repetitive work, better access to technical knowledge, improved documentation, and faster debugging and test creation.

4. Technical debt
Technical debt is the future maintenance cost associated with shortcuts, poor design, duplicated code, missing tests, or outdated dependencies. Examples include complex functions, repeated code, poorly documented modules, obsolete libraries, and inadequate tests.

5. AI-based technical debt prediction
AI can analyze repository history and code-quality metrics to identify modules likely to become difficult or expensive to maintain. Useful indicators include code complexity, duplication, change frequency, defect history, dependency age, test coverage, and coupling between modules.

6. Prioritizing debt
Combine predicted risk with business importance, maintenance cost, and available resources. For example, a frequently modified payment module with complex logic and recurring defects may deserve earlier attention than a rarely changed utility.

7. Challenges
Predictions may be inaccurate; metrics do not fully represent code quality; AI may suggest unnecessary refactoring; generated code can introduce maintenance problems; and developers must avoid excessive dependence on AI.

Conclusion: Human–AI collaboration improves development when people verify AI output. Technical-debt prediction helps teams identify and prioritize maintenance rather than relying only on intuition.

Q5. Explain AI for developer productivity analytics, including benefits, challenges, and research trends.

Introduction: Developer productivity analytics uses information about software workflows to understand bottlenecks, improve collaboration, and reduce unnecessary work. AI can analyze these patterns and suggest process improvements.

1. Productivity analytics
It examines build duration, test execution time, pull-request review time, deployment frequency, defect and incident trends, and time spent waiting for automated processes. Indicators should be interpreted collectively rather than used alone to judge individuals.

2. Role of AI
AI can identify repetitive tasks suitable for automation, summarize pull requests, suggest code and documentation, identify slow build and test stages, detect recurring bottlenecks, forecast delivery risks, and recommend workflow improvements.

3. Benefits
Reduced waiting time, faster feedback, improved collaboration, better use of development resources, more informed process decisions, and potentially faster delivery of reliable software.

4. Challenges
- **Misleading metrics:** More commits do not necessarily mean higher-quality work.
- **Privacy:** Detailed monitoring may reveal sensitive employee information.
- **Bias:** Models may evaluate different types of work unfairly.
- **Overdependence:** Developers may accept suggestions without verification.
- **Security:** Confidential code and development data must be protected.

5. Responsible use
Use team-level indicators where appropriate, explain metrics, protect data, combine quantitative measures with human judgment, measure quality alongside speed, and evaluate tools for unintended consequences.

6. Research trends
Important research directions include more reliable AI coding assistants, automated test generation and repair, AI-supported maintenance, explainable defect prediction, secure AI-assisted development, human–AI collaboration, and evaluation of generated software quality. Results depend on the tools and tasks evaluated.

Conclusion: AI-driven productivity analytics can improve workflows, but it should focus on removing bottlenecks and improving software outcomes rather than reducing performance to simplistic numerical scores.

PART B — 10 TWO-MARK QUESTIONS WITH ANSWERS

1. What is Explainable AI (XAI)?

Explainable AI refers to AI systems whose decisions or predictions can be understood by humans. It helps developers interpret recommendations and investigate errors.

2. Define transparency in AI.

Transparency means providing understandable information about an AI system's purpose, data, operation, limitations, and use.

3. What is bias in AI?

AI bias occurs when a system produces systematically unfair or unrepresentative results, often because of biased data, design choices, or evaluation methods.

4. Mention two ethical issues in AI-driven software engineering.

Two ethical issues are privacy violations and biased decision-making. Other concerns include accountability, reliability, and intellectual-property risks.

5. What is risk management in AI?

Risk management is the process of identifying, assessing, reducing, and monitoring potential problems caused by AI systems.

6. Define technical debt.

Technical debt is the future cost of maintaining software because of shortcuts, poor design, duplicated code, or outdated dependencies.

7. What is human–AI collaboration?

Human–AI collaboration combines human expertise and judgment with AI assistance for coding, testing, debugging, and documentation.

8. What is developer productivity analytics?

It uses development workflow data to identify bottlenecks and improve processes, collaboration, and software delivery.

9. What are two legal concerns associated with AI-generated code?

Two concerns are copyright and software licensing compliance. Other concerns include privacy, contractual obligations, and applicable regulations.

10. Why is human oversight important in autonomous code deployment?

Human oversight helps verify risky changes, prevent harmful automated actions, ensure compliance, and make informed decisions when failures occur.